Not all dependencies are equal:
Intervener complexity across dependency types in natural languages

Project id: LE1

CGS410 course project | Shlesha Chawla
https://github.com/shlesha1804/CGS410_ShleshaChawla

1. **Motivation for the research problem:**
   In natural language sentences, words that are grammatically related are not always adjacent to each other. For example, in the sentence "The brown dog always quickly chased the small cat," the subject *dog* and the verb *chased* are separated by two adverbs. The words that appear between a head and its dependent are called **interveners**.
   Prior research on dependency length minimization has shown that languages tend to keep related words close together to reduce memory load during sentence processing. However, most existing work treats all dependency types uniformly, asking only *how long* dependencies are, not *what kinds* of material appear between them. This project addresses that gap: not all dependency relations may be equally tolerant of complex intervening material.
   The **research question** is: What kind of words typically appear between two related words that are not next to each other in natural language sentences. Does this also depend on the kind of dependency? Are object-verb(obj) or adjective-noun(amod) dependencies more tolerant of complex intervening material than subject-verb(nsubj) dependencies? Answering these questions advances our understanding of how grammatical relations shape the structure of sentences across languages.

2. **Hypothesis and Prediction:**
   The central hypothesis is that *intervener complexity is not uniform across dependency types*. Specifically, I predict the following:

H1	Tight dependencies (det, amod) will have low complexity interveners, i.e, low arity, small subtree size, and high proportion of function words. (det: determiner-noun)
H2	Core argument dependencies (nsubj, obj) will have moderately complex interveners, but nsubj will be more restrictive than obj.
H3	Adjunct dependencies (advmod, obl) will tolerate the most structurally complex intervening material.(advmod: adverbial modifier ; obl:oblique nominal)
H4	These patterns will hold consistently across typologically diverse languages, suggesting a universal grammatical tendency.

3. Methods:

3.1 Data

I used treebanks from the Universal Dependencies project, covering 9 typologically diverse languages: English, Hindi, Arabic, Turkish, Japanese, Chinese, French, Spanish, and German. These languages represent a wide range of word order typologies, morphological complexity and geographic families. A total of **1,673,851 intervener** records were extracted from across all languages**.**

3.2 Dependency Types Analyzed

focused on **seven** Universal Dependencies relation types: **nsubj** (nominal subject), **obj** (direct object), **amod** (adjectival modifier), **det** (determiner), **advmod** (adverbial modifier), **obl** (oblique nominal), and **nmod** (nominal modifier). These represent a broad cross-section of grammatical relations from tight nominal dependencies (such as: det) to looser adjunct relations (such as: advmod, obl).

3.3 Intervener Feature Extraction

For each dependency arc (head, dependent) where the two words were not linearly adjacent, I identified all tokens appearing between them and recorded the following features for each intervener:

Feature	Description
intervener_upos	Universal POS tag (NOUN, VERB, ADJ, etc.)
intervener_arity	Number of direct dependents of the intervener
intervener_subtree_size	Total number of nodes in the intervener's subtree (self + all descendants)
intervener_heads_structure	Binary: 1 if subtree_size > 1 (i.e., it governs further material), else 0
dep_length	Linear distance between head and dependent (in tokens)

3.4 Inference Method

I compared intervener complexity across dependency types using descriptive statistics: mean arity, mean subtree size, and the percentage of interveners that head further syntactic structure. To test H1-H3, we ranked dependency types by these metrics and compared rankings across all 9 languages. Consistency of a pattern across languages was taken as evidence of a cross-linguistically robust tendency. The dependency types were grouped into three tiers

a. tight (det, amod),
b. argument (nsubj, obj)
c. adjunct (advmod, obl, nmod)

4. Results:

4.1 Overall Intervener Complexity by Dependency Type

Table 1 shows the mean intervener arity and subtree size collapsed across all 9 languages. A clear three-tier pattern emerges, consistent with the predictions.

Dependency	Mean Arity	Mean Subtree Size	% Heads Structure	Tier
det	0.229	1.336	15.4%	Tight
amod	0.277	1.389	18.0%	Tight
nmod	0.282	1.455	16.6%	Modifier
obj	0.405	1.607	23.3%	Argument
nsubj	0.556	1.986	29.5%	Argument
advmod	0.523	1.990	28.5%	Adjunct
obl	0.503	1.913	27.8%	Adjunct

Table 1: Mean intervener complexity by dependency type, averaged across 9 languages.

The results strongly support H1: det and amod have the lowest intervener complexity on all three metrics. Their interveners are predominantly function words and single-token modifiers with no further dependents. For example, in English, 43% of det interveners are adjectives (ADJ), which are structurally simple leaves.

For H2, nsubj consistently shows higher intervener complexity than obj across all languages. In Hindi, nsubj has a mean subtree size of 2.258 vs. obj's 2.157. In English, nsubj arity (0.287) exceeds obj arity (0.162). This asymmetry suggests that the subject position tolerates more complex intervening material than the object position, possibly because subjects are typically sentence-initial and more material must cross the subject-verb boundary.

For H3, advmod and obl show high intervener complexity, but interestingly nsubj rivals them in several languages. This suggests that the subject-verb dependency is not as tight as traditionally assumed. It patterns more like an adjunct relation than a tight nominal one in terms of intervener complexity.

4.2 Cross-Linguistic Consistency

The three-tier pattern (det/amod < obj/nmod < nsubj/advmod/obl) holds across all 9 languages, supporting H4. However, the absolute values differ substantially. Arabic and Japanese show the highest overall intervener complexity (Arabic advmod mean subtree size: 2.657; Japanese advmod: 2.421), while French and English show the lowest (French det subtree size: 1.147). Turkish has relatively high intervener complexity even for tight dependencies like det (arity: 0.360), likely due to its highly inflected word structure and flexible sentence ordering, which together allow more material to appear between related words.

4.3 Intervener POS Distributions

The POS tag distributions of interveners also differ systematically by dependency type. For det dependencies, interveners are dominated by adjectives (ADJ), these are pre-nominal modifiers sitting between a determiner and its noun. For nsubj and obl, interveners are predominantly nouns (NOUN) and adpositions (ADP), indicating that full nominal phrases and prepositional phrases cross these dependencies. For obj in Japanese, 69% of interveners are ADP (postpositions), reflecting the verb-final SOV structure where case markers systematically appear between objects and verbs.

5. Theoretical Implications:

For dependency length minimization (DLM) theory: Prior work assumes that all dependencies are equally costly to maintain. The results suggest that the kind of dependency matters: tight dependencies like det actively resist complex intervening material, while adjunct relations like obl tolerate it. DLM models should incorporate dependency-type weights rather than treating all arcs uniformly.

For sentence processing: The higher complexity of nsubj interveners may explain why subject-verb agreement errors are more common when intervening material is complex. The intervener material for nsubj is structurally heavier, creating more potential for memory interference during agreement computation.

For language universals: The three-tier pattern (tight nominal < argument < adjunct) is consistent across SOV, SVO, and VSO languages, and across analytic and synthetic

morphological types. This suggests it reflects a deep grammatical universal: the more peripheral a dependency relation is to the core predicate-argument structure, the more complex the material it tolerates between its endpoints.

For language evolution: If intervener complexity constraints are universal, they may reflect functional pressures that shaped the evolution of grammatical relations themselves, tighter relations evolved precisely because they need to be processed reliably under memory constraints, while adjunct relations could remain looser because they contribute optional rather than obligatory information.

References:

- **Nivre, J. et al. (2020). Universal Dependencies v2. *LREC 2020*.**
- **Gibson, E. (1998). Linguistic complexity: Locality of syntactic dependencies.**
- **Futrell, R., Mahowald, K., & Gibson, E. (2015). Large-scale evidence of dependency length minimization in 37 languages.**
- **Liu, H. (2008). Dependency distance as a metric of language comprehension difficulty.**
- **https://universaldependencies.org/**
- **English (EWT): https://github.com/UniversalDependencies/UD_English-EWT**
- **Hindi (HDTB): https://github.com/UniversalDependencies/UD_Hindi-HDTB**
- **Arabic (PADT): https://github.com/UniversalDependencies/UD_Arabic-PADT**
- **Turkish (IMST): https://github.com/UniversalDependencies/UD_Turkish-IMST**
- **Japanese (GSD): https://github.com/UniversalDependencies/UD_Japanese-GSD**
- **Chinese (GSD): https://github.com/UniversalDependencies/UD_Chinese-GSD**
- **French (GSD): https://github.com/UniversalDependencies/UD_French-GSD**
- **Spanish (GSD): https://github.com/UniversalDependencies/UD_Spanish-GSD**
- **German (GSD): https://github.com/UniversalDependencies/UD_German-GSD**

Appendix: Code

The full implementation is available on GitHub
https://github.com/shlesha1804/CGS410_ShleshaChawla

Below are the core functions. The pipeline consists of four modules: loader.py, extractor.py, analyzer.py, and visualizer.py, orchestrated by main.py.

A.1 Intervener Extraction (extractor.py)

```python
def extractinterveners(
    sentences, language: str, target_deps: List[str]) -> List[Dict[str, Any]]:
    records = []

    for sentid, sentence in enumerate(sentences):
        tokbyid = {tok["id"]: tok for tok in sentence}


        children = childrenmapping(sentence)

        for tok in sentence:
            deprel = tok["deprel"]

            basedeprel = deprel.split(":")[0]

            if basedeprel not in target_deps:
                continue

            depid = tok["id"]
            headid = tok["head"]


            if headid == 0 or headid not in tokbyid:
                continue


            left = min(depid, headid)
            right = max(depid, headid)
            dep_length = right - left


            if dep_length <= 1:
                continue


            intervenerids = list(range(left + 1, right))

            for iv_id in intervenerids:
                if iv_id not in tokbyid:
                    continue
                iv_tok = tokbyid[iv_id]

                arity = len(children.get(iv_id, []))
                subsize = subtreesize(iv_id, children)
                heads_structure = subsize > 1

                records.append({
                    "language":                  language,
                    "sent_id":                   sentid,
                    "deprel":                    basedeprel,
                    "head_id":                   headid,
                    "dep_id":                    depid,
                    "dep_length":                dep_length,
                    "intervener_id":             iv_id,
                    "intervener_form":           iv_tok["form"],
                    "intervener_upos":           iv_tok["upos"],
                    "intervener_arity":          arity,
                    "intervener_subtree_size":   subsize,
                    "intervener_heads_structure": int(heads_structure),
                })

    return records
```

A.2 Statistical Aggregation (analyzer.py)

```python
def computestats(records: List[Dict[str, Any]]) -> pd.DataFrame:
    df = pd.DataFrame(records)

    agg = (
        df.groupby(["language", "deprel"])
        .agg(
            n_pairs           = ("dep_length", "count"),
            mean_dep_length   = ("dep_length", "mean"),
            median_dep_length = ("dep_length", "median"),
            mean_arity        = ("intervener_arity", "mean"),
            mean_subtree_size = ("intervener_subtree_size", "mean"),
            pct_heads_struct  = ("intervener_heads_structure", "mean"),
        )
        .reset_index()
    )
```